

第9讲

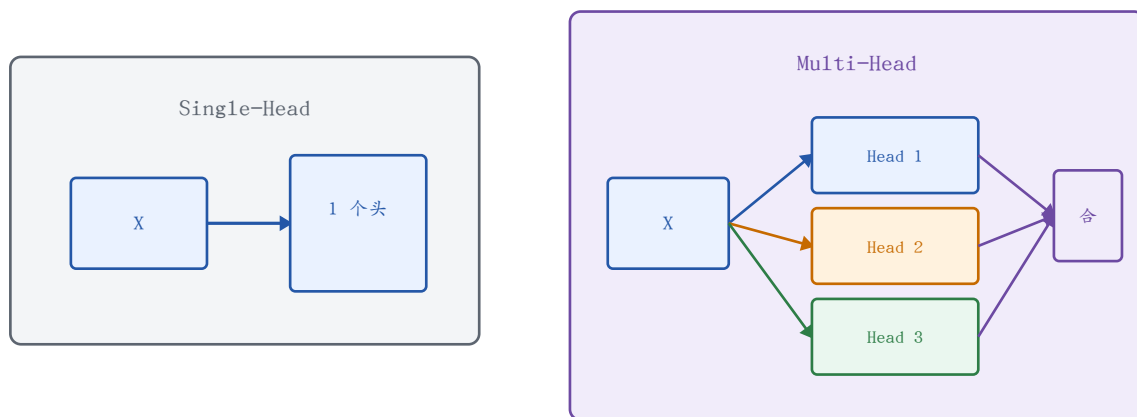
Multi-Head Attention：为什么一个注意力头不够？

从多个子空间，到 $\text{Concat} + W_0$ ，再到 GQA/MQA 的最小直觉

本讲目标

学完后，你应该能解释：Head 到底是什么；为什么同一个输入要同时做多套 Q/K/V； d_{model} 、 h 、 d_{head} 如何对应；不同 Head 如何并行计算；Concat 和 W_0 各自做什么；以及为什么“多个头看不同关系”只是直觉而不是固定规则。

Single-Head 与 Multi-Head：区别不只是“算多次”



多个头使用不同的投影参数，因此可以形成不同的匹配与读取子空间

先记一句话

Multi-Head Attention = 在多个不同的可学习子空间里并行做 Attention，再把这些结果合并。

1. 回顾：单头 Attention

已经能看全句，为什么还不够？

上一讲里，一个 Query 可以和所有 Key 比较，再按权重读取 Value。乍看之下，一个头已经能“看到所有 token”，似乎没有必要再做多个头。

问题的核心

“能看到所有位置”不等于“能同时用多种不同的表示方式去比较这些位置”。单个 Head 只有一套 W_Q 、 W_K 、 W_V ，它被限制在这一套投影形成的子空间里。

类比：同一篇论文，你可以同时从“研究主题”“实验方法”“数据集”“结论可靠性”四个角度看。一个角度可以浏览全文，但未必适合同时候把所有关系都表达得很好。

2. Head 到底是什么？

一个 Attention Head，可以理解成一套独立的 Attention 计算通道。它拥有自己的 Q/K/V 投影参数，并在较小的向量维度里完成 Attention。

```
head_i = Attention(Q_i, K_i, V_i)
```

其中：

```
Q_i = X W_Q^(i) K_i = X W_K^(i) V_i = X W_V^(i)
```

重要

Head 不是一个 token，也不是一个独立小模型。它是同一层 Attention 内的一条并行计算分支。

你可能的误解	正确理解
“8 个 Head = 8 个模型”	不是；仍是一层内部的并行分支
“每个 Head 看不同 token”	不一定；通常每个 Head 都能访问允许范围内的全部位置
“Head 数越多一定越强”	不成立；需要维度、参数、训练与架构共同权衡

3. 多头的真正差别：不同的投影矩阵

假设有 8 个 Head。并不是把同一套 Q/K/V 复制 8 份，而是让不同 Head 具有不同的参数。

Head	Q 投影	K 投影	V 投影
Head 1	$W_Q^{(1)}$	$W_K^{(1)}$	$W_V^{(1)}$
Head 2	$W_Q^{(2)}$	$W_K^{(2)}$	$W_V^{(2)}$
...	...	...	...
Head h	$W_Q^{(h)}$	$W_K^{(h)}$	$W_V^{(h)}$

底层原理

不同参数意味着：即使输入 X 完全相同，不同 Head 得到的 Q/K/V 也不同，因此算出的相关性分数、注意力权重和输出都可能不同。

4. 为什么“不同子空间”这个说法很重要？

原始表示 x 可能包含许多混在一起的信息。每个 Head 的线性投影，可以把它映射到一个更小、更专门的表示空间。

“不同头看不同关系”是有用直觉，但不是硬编码规则



真实训练中这些功能由数据和优化共同形成，不是程序员预先指定 Head A 必须负责某种语法关系

科研上要谨慎

“Head A 负责语法、Head B 负责代词”适合建立直觉，但真实模型中 Head 的功能可能重叠、分布式、难解释，也可能出现冗余。不要把教学比喻当成硬事实。

5. 一个生活类比：同一个会议，同时请不同专家听

假设一段多 Agent 对话很长。你请四个人同时听：

- 专家 A 只关注“谁提出了任务目标”；
- 专家 B 重点追踪“谁反驳了谁”；
- 专家 C 重点找数字、条件和约束；
- 专家 D 重点关注当前行动与前文计划的对应关系。

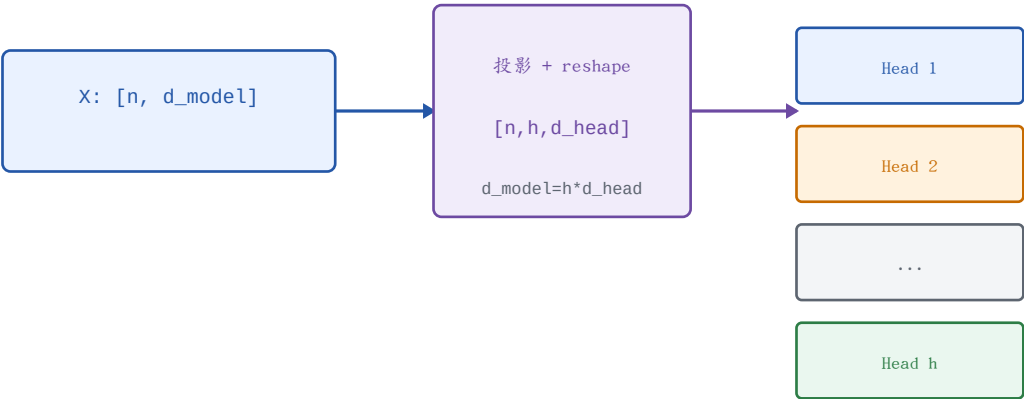
他们听的是同一段对话，但“关注标准”和“提取内容”不同。最后再把四份摘要合并。

这正对应 Multi-Head 的直觉

同一个 X $\rightarrow$ 不同投影 $\rightarrow$ 不同 Attention 模式 $\rightarrow$ 多份输出 $\rightarrow$ 合并。

6. d_model、h、d_head：三个维度先认清

常见做法：把 d_model 拆成 h 个 head，每个 head 的维度更小



例：d_model=512，h=8，则典型 d_head=64（用于理解；真实模型可采用其他配置）

符号	含义	例子
d_model	模型隐藏表示总维度	512
h	Head 数量	8
d_head	每个 Head 的 Q/K/V 维度	64
常见关系	$d_{\text{model}} = h \times d_{\text{head}}$	$512 = 8 \times 64$

注意

“每头 64 维”是经典教学例子，不是所有模型必须如此。不同架构可能采用不同 head 数、Q/K/V 维度或共享方式。

7. 为什么不是每个 Head 都保留完整 d_{model} ?

如果有 h 个 Head，而每个 Head 都用完整 d_{model} 维度，计算量和参数量会明显膨胀。经典 Multi-Head 的一个关键设计，是让每个 Head 在较低维度工作，然后拼接回来。

直觉

与其让 8 个专家每人复制一整套超大办公室，不如把总预算分成 8 个小组，各自在不同方向工作，最后汇总。

这样可以在总体表示宽度近似固定的情况下，增加并行“视角”。

8. Shape 是理解 Multi-Head 的捷径

先忽略 batch。假设序列长度 $n=10$, $d_{model}=512$, $h=8$, $d_{head}=64$ 。

阶段	Shape	解释
输入 X	[10,512]	10 个 token, 每个 512 维
Q/K/V 投影	各 [10,512]	一次大矩阵投影后再重排
拆 Head	[8,10,64]	8 个头, 每头 10 个 token $\times$ 64 维
每头权重	[8,10,10]	每个头各有一个 10×10 Attention Matrix
每头输出	[8,10,64]	每头给每个 token 64 维输出
Concat	[10,512]	8×64 再拼回 512 维
乘 W_O	[10,512]	重新混合成层的最终 Attention 输出

看到论文时

如果公式很复杂，先写 Shape。很多“看不懂”的 Attention 公式，一旦维度对齐，逻辑会清晰很多。

9. 每个 Head 内部仍然是上一讲那套公式

Multi-Head 并没有发明另一种 Attention。每个 Head 内部仍然是：

```
head_i = softmax(Q_i K_i^T / sqrt(d_head)) V_i
```

变化只在于：我们并行得到多套 Q_i 、 K_i 、 V_i 。

- 每个 Head 单独计算 QK^T ;
- 每个 Head 单独做 Causal Mask (如果是 GPT 类自回归模型);
- 每个 Head 单独做 Softmax;
- 每个 Head 单独加权自己的 V 。

因此

学懂单头 Attention 后, 多头 Attention 的难点其实主要变成了: 如何分头、如何合头、为什么值得分头。

10. 多个 Head 可以有不同 Attention Matrix

假设句子有 6 个 token。Head 1 和 Head 2 都会产生 6×6 的 Attention Matrix，但矩阵里的权重可能完全不同。

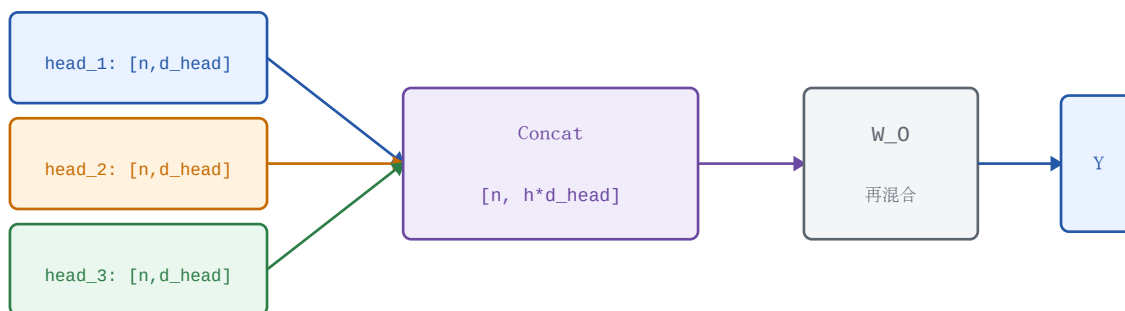
Head	可能形成的权重模式（只是示意）
Head 1	更偏向邻近 token，局部连接较强
Head 2	跨较远位置建立较强连接
Head 3	某些标点、边界或结构位置权重较强
Head 4	形成难以直接用人类概念命名的模式

为什么会不同？

因为每个 Head 的 W_Q/W_K 不同。同样的输入经过不同投影后，点积结构就不同。

11. Concat: 先把各 Head 的输出拼起来

多个 Head 的输出如何重新合并?



`Concat` 只是拼接; `W_0` 才让各 Head 的信息重新线性混合回模型表示空间

假设 8 个 Head, 每个输出 64 维:

```
head_1, ..., head_8: each [n,64] Concat(head_1,...,head_8): [n,512]
```

`Concat` 只是拼接

它不会自动让 Head 之间“互相理解”。它只是把多份特征并排放回一个长向量。

12. W_O : 为什么拼接以后还要再乘一个矩阵?

如果只做 Concat, 那么每个 Head 的信息只是机械地占据不同维度区间。输出投影 W_O 允许模型重新混合这些维度。

```
MultiHead(X) = Concat(head_1,...,head_h)  $W_O$ 
```

类比

四个专家各写一页意见。Concat 相当于把四页订在一起; W_O 更像主编把四份意见重新整合成统一格式的最终报告。

W_O 也是可学习参数, 会和其他权重一起通过反向传播更新。

13. 完整 Multi-Head Attention 公式

Multi-Head Attention 完整流水线



$$\blacksquare \blacksquare \text{Head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_0$$

关键：各头使用不同投影参数，因此不是同一件事机械复制 h 次

经典写法可以压缩为：

$$\text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V) \quad \text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

记忆口令

先各自投影 $\rightarrow$ 各头独立 Attention $\rightarrow$ 拼接 $\rightarrow$ W_0 再混合。

14. 参数到底增加在哪里？

初学者会担心：8 个 Head 是不是把参数直接乘 8？经典设置下不一定，因为每个 Head 的维度相应缩小。

如果 $d_{model} = h \times d_{head}$ ，那么把各头 $W_Q^{(i)}$ 看作拼起来，整体上等价于一个 $d_{model} \times d_{model}$ 的 Q 投影矩阵；K、V 类似。

模块	典型参数规模（忽略 bias）
Q 投影	$d_{model} \times d_{model}$
K 投影	$d_{model} \times d_{model}$
V 投影	$d_{model} \times d_{model}$
输出 W_O	$d_{model} \times d_{model}$
合计	约 $4 \times d_{model}$

核心

多头的价值主要不是“简单增加总维度”，而是把表示组织成多个并行子空间。

15. Multi-Head 为什么可能比 Single-Head 表达更丰富？

一个 Head 必须用同一套 Q/K/V 投影同时处理许多关系。多个 Head 则允许不同参数共同分担。

- 不同 Head 可以形成不同的相似度几何；
- 不同 Head 可以对同一对 token 给出不同关注强度；
- 不同 Head 的 V 投影也不同，因此“读出来的内容”不同；
- W_O 可以把这些不同视角重新组合。

更底层地说

Multi-Head

给模型增加了多组并行的内容寻址机制。它不是简单多数投票，而是多组连续向量变换共同构造新表示。

16. “多个头负责不同语义”是真的吗？

部分研究确实观察到某些 Head 与特定位置、句法或依赖模式相关，但不能把这种现象过度简化。

说法	判断
“有些 Head 会形成可解释模式”	可以发生
“每个 Head 都有一个清晰固定职责”	不成立
“头与头一定完全不同”	不成立，可能冗余
“看 Attention 图就知道模型因果机制”	不成立，需要更严格的干预分析

科研提醒

以后如果你研究 Agent / LLM 内部机制，不要只凭一张 Attention Heatmap 下机制结论。可视化是线索，不等于因果证据。

17. Head 数是不是越多越好？

不是。Head 数变化会同时改变每头维度、计算布局、表示能力和硬件效率。

- Head 太少：可并行的表示子空间较少；
- Head 太多但 `d_model` 固定：每个 Head 可能变得很窄；
- 过多 Head 可能出现冗余，未必带来收益；
- 实际架构选择还要考虑 GPU/TPU 的矩阵尺寸与并行效率。

不要形成机械规律

不能写成“16 个头一定比 8 个头强”。这是架构超参数，需要和模型规模、训练数据、任务、实现共同看。

18. Causal Multi-Head Attention: GPT 每个 Head 都不能偷看未来

在 GPT 类自回归模型中，每个 Head 的 Attention Matrix 都要遵守同一个因果约束：当前位置 i 只能看位置 $\leq i$ 。

```
scores_i = Q_i K_i^T / sqrt(d_head) scores_i[j>position] = -infinity weights_i = softmax(scores_i)
```

关键

Mask 不是只加一次“给整个模型看”。在实现上，它会作用到每个 Head 的 score 矩阵，使所有 Head 都遵守不看未来的规则。

19. Multi-Head

的计算是“并行”还是“一个头一个头算”？

概念上我们常说 Head 1、Head 2、Head 3 分别计算；但高效实现通常不会写 Python 循环逐头慢慢算，而是把 Head 维度加入张量，一次性用批量矩阵乘法并行计算。

```
Q shape: [batch, h, n, d_head] K shape: [batch, h, n, d_head] Q @ K^T -> [batch, h, n, n]
```

为什么这很重要？

现代深度学习系统的速度，不只取决于数学公式，还取决于能否把计算组织成 GPU 擅长的大矩阵/批量矩阵运算。

20. 一个极小数值例子：两个 Head 得到不同结果

假设同样有 3 个 token。为了教学简化，我们直接给两个 Head 不同的 Q/K/V，不再展开 $W_Q/W_K/W_V$ 的矩阵乘法。

Head	Attention 权重示意	输出直觉
Head 1	[0.75, 0.20, 0.05]	主要读取 token 1
Head 2	[0.10, 0.25, 0.65]	主要读取 token 3

即使两个 Head 看到同样的 3 个 token，也可以因为投影不同而形成完全不同的权重。最后把两份输出拼接，再经 W_O 混合。

本讲 Python 实验

配套代码会真正打印两个 Head 的 scores、weights、head output、Concat 结果以及 W_O 后的最终输出。

21. Multi-Head 和 “集成学习 / 多 Agent 投票” 不是一回事

因为你正在学 Agent Systems，很容易把多个 Attention Head 类比成多个 Agent。但底层机制完全不同。

Multi-Head Attention	多 Agent 系统
一个神经网络层内部的并行向量计算	多个相对独立的推理/行动实体
共同参与同一次前向传播	可能有独立上下文、记忆、工具和通信
参数通过同一个训练目标联合学习	Agent 之间可以同模型或异模型、同策略或异策略
输出是连续向量再混合	输出常是消息、动作、方案或投票

可以类比，但不能等同

“多个视角后融合” 这个抽象思想相似；实现层级和研究问题完全不同。

22. 扩展：为什么后来又出现 MQA 和 GQA?

扩展：MHA / MQA / GQA 的 K/V 共享方式



这里只建立概念：后面讲 KV Cache 和推理系统时再深入“为什么共享 K/V 可以省内存与带宽”

标准 MHA 中，每个 Query Head 通常都有对应的 K/V Head。推理时，历史 token 的 K/V 需要缓存。K/V 组越多，KV Cache 越大。

先建立直觉

MQA 让很多 Q Head 共享一组 K/V；GQA 让若干 Q Head 共享一组 K/V。这样可以减少 KV Cache 和内存带宽压力。代价与效果是架构权衡。

这里不要求掌握工程细节。后面讲 KV Cache 时，我们会重新回来。

23. MHA / MQA / GQA：只记最小区别

结构	Query Heads	Key/Value Heads	核心直觉
MHA	多个	通常与 Q 头数量对应	每头各有 K/V
MQA	多个	1 组	所有 Q 头共享 K/V
GQA	多个	少于 Q 头、多于 1	Q 头分组共享 K/V

不要提前陷入细节

你现在只需要知道：“多头”这个概念不仅影响表达，也会影响推理时 K/V 的存储和系统成本。

24. 常见误区集中纠正

误区	正确理解
Head = 独立神经网络	不是；是一层 Attention 内的并行分支
每个 Head 只看句子的一部分 token	通常每个 Head 都可看允许范围内全部位置
不同 Head 是手工指定不同任务	通常不是，由训练学习形成
Head 越多越强	没有这种单调规律
Concat 已经完成融合	Concat 只是拼接， W_O 进一步混合
Multi-Head = 多 Agent	不是，层级完全不同
看 Heatmap 就能解释模型	只能作为线索，不能自动成为因果解释

25. 本讲总结：把 Multi-Head 压缩成 8 句话

Multi-Head Attention 完整流水线



$$\blacksquare \text{Head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

关键：各头使用不同投影参数，因此不是同一件事机械复制 h 次

- 一个 Head 是一套独立的 Q/K/V 投影 + Attention 计算通道。
- 多个 Head 共享同一个输入 X，但投影参数不同。
- 经典设计把 d_{model} 分成 h 个较小的 d_{head} 。
- 每个 Head 内部仍然执行 Scaled Dot-Product Attention。
- 不同 Head 可以形成不同的 Attention Matrix 和 Value 表示。
- 所有 Head 输出先 Concat。
- 再通过可学习的 W_O 把多头信息重新混合回模型空间。
- MQA/GQA 通过共享 K/V Head 减少推理时的缓存与带宽压力。

完整公式

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

26. 术语表

术语	英文	零基础解释
注意力头	Attention Head	一套独立 Q/K/V 投影和 Attention 分支
多头注意力	Multi-Head Attention	多个 Head 并行 Attention 后再合并
模型维度	d_model	token 隐藏表示的总宽度
头维度	d_head	单个 Head 内 Q/K/V 的向量宽度
头数	h	并行 Attention Head 的数量
拼接	Concatenation / Concat	把多个 Head 输出在特征维拼起来
输出投影	Output Projection / W_O	对拼接结果做可学习线性混合
多查询注意力	Multi-Query Attention / MQA	多个 Q 头共享一组 K/V
分组查询注意力	Grouped-Query Attention / GQA	若干 Q 头按组共享 K/V
冗余 Head	Redundant Head	功能重叠、贡献可能相似的 Head
批量矩阵乘	Batched MatMul	GPU 一次并行处理多个 Head 的常见实现方式

27. 练习题（先自己做）

- 1. 用一句话解释 Attention Head 是什么。
- 2. Multi-Head Attention 与把同一个 Attention 结果复制多份有什么根本区别？
- 3. 为什么不同 Head 会产生不同的 Attention Matrix？
- 4. d_{model} 、 h 、 d_{head} 分别表示什么？
- 5. 若 $d_{model}=768$ ， $h=12$ ，在经典等分设置下 d_{head} 是多少？
- 6. 输入 $X=[n, d_{model}]$ ，拆成多头后常见 Shape 是什么（忽略 batch）？
- 7. 每个 Head 内部执行的核心 Attention 公式是什么？
- 8. 为什么说“不同 Head 学不同语法角色”只能作为直觉？
- 9. Concat 做了什么？它是否已经完成 Head 间的信息融合？
- 10. W_O 的作用是什么？
- 11. 写出 MultiHead 的经典公式，并逐项解释。
- 12. 为什么经典多头设计不会简单把总表示维度乘 h ？
- 13. Head 数是否越多越好？为什么？
- 14. GPT 中 Causal Mask 如何作用于多头 Attention？
- 15. 为什么高效实现不会真的用 Python for 循环逐个 Head 计算？
- 16. Multi-Head Attention 和多 Agent 投票有什么本质区别？
- 17. MHA、MQA、GQA 在 K/V Head 数量上的区别是什么？
- 18. 为什么减少 K/V Head 数量有助于推理系统？
- 19. 如果一个模型 Attention Heatmap 显示某个 Head 总关注句号，能否直接断言“这个 Head 的功能就是识别句号”？为什么？
- 20. 请画出完整流程： $X \rightarrow Q/K/V \rightarrow \text{Split Heads} \rightarrow \text{每头 Attention} \rightarrow \text{Concat} \rightarrow W_O \rightarrow \text{Output}$ 。

28. 练习题答案与自检

第1题

一条拥有自己 Q/K/V 投影参数、在较小维度内独立执行 Attention 的并行计算分支。

第2题

不同 Head 使用不同可学习投影矩阵，因此得到不同 Q/K/V、权重和输出，不是同一结果的复制。

第3题

因为各 Head 的 W_Q/W_K 不同，同一输入被映射到不同子空间后，点积分数自然不同。

第4题

d_{model} 是总隐藏维度； h 是 Head 数； d_{head} 是每个 Head 的 Q/K/V 维度。

第5题

$768/12=64$ 。

第6题

常写成 $[h,n,d_{head}]$ ；含 batch 时常见 $[batch,h,n,d_{head}]$ 。

第7题

$\text{softmax}(Q_i K_i^T / \sqrt{d_{head}}) V_i$ ；GPT 类模型还在 Softmax 前加因果 Mask。

第8题

真实 Head 功能由训练涌现，可能重叠、冗余或难命名，不是人为硬编码一个 Head 对应一种语法关系。

第9题

Concat 把多个 Head 的特征在最后一维拼起来；它本身只是并排放置，不是充分混合。

第10题

W_O 对拼接后的多头表示做可学习线性变换，使不同 Head 的信息重新混合并回到模型表示空间。

第11题

$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ ； $\text{MultiHead} = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$ 。

第12题

经典设计在 `d_model` 固定时把它分给多个较小 `d_head`，拼接后仍回到 `d_model`。

第13题

不是；头太多会使单头变窄，可能冗余，还受参数、训练和硬件效率影响。

第14题

同一个因果约束会作用到每个 Head 的 score 矩阵，使未来位置在所有 Head 中权重为 0。

第15题

GPU 擅长批量矩阵运算；把 Head 作为张量维度可并行完成，远快于解释器层循环。

第16题

Multi-Head 是单个神经网络层内部的连续向量分支；多 Agent 是多个相对独立推理/行动实体，可能有独立记忆、工具和通信。

第17题

MHA 通常多 Q 头对应多 K/V 头；MQA 多 Q 头共享 1 组 K/V；GQA 多 Q 头分组共享较少数量 K/V。

第18题

历史 token 的 K/V 需要缓存；K/V 头更少意味着 KV Cache 和内存带宽压力更低。

第19题

不能。它只是观察到相关模式，不足以证明因果功能；需要干预、消融或其他机制分析。

第20题

自检图应包含： $X \rightarrow$ 线性 Q/K/V 投影 $\rightarrow$ reshape/split 为 h 头 $\rightarrow$ 每头 scaled dot-product Attention $\rightarrow$ Concat $\rightarrow$ W_O $\rightarrow$ 输出。

29. 你已经能读 Transformer 原论文中的哪一块？

到这一讲，你已经具备理解 Transformer 原始架构中 Multi-Head Attention 核心公式的最低闭环：你知道 Q/K/V 如何产生、每个 Head 内部如何做 Attention、为什么拆头，以及为什么最后需要 Concat + W_O。

看到的符号	你现在应该想到什么
W_i^Q, W_i^K, W_i^V	第 i 个 Head 的三套投影参数
$head_i$	第 i 个 Head 的 Scaled Dot-Product Attention 输出
$Concat(head_1, \dots, head_h)$	把各头输出拼回总特征维
W^O	输出投影，重新混合多头信息
h	Head 数，不是 Agent 数

下一讲

第10讲进入 位置编码：Transformer 为什么必须额外告诉模型“谁在前、谁在后”？我们会从“Attention 本身对排列顺序不敏感”的问题出发，讲绝对位置编码、正弦位置编码，以及现代 LLM 常见的 RoPE 直觉。